

Socioeconomic Drivers of Climate Change

Multi-Target ML Analysis (1900–2023)

2–3 Minute Video Presentation Script with Notebook Cell References

Abdul Moiz Hussain | Muhammed Abdullah Khan | May 2026



Presentation Timing Overview

Section	Notebook	Key Cells to Show	Time
Introduction & Problem Statement	—	README / Title Slide	0:00 – 0:20
Exploratory Data Analysis	Notebook 1	Cells 2, 3, 4, 5, 6, 7, 11, 14	0:20 – 0:55
Preprocessing Pipeline	Notebook 2	Cells 2, 3, 6	0:55 – 1:25

Section	Notebook	Key Cells to Show	Time
Model Training & Tuning	Notebook 3	Cells 1, 4, 5, 7, 9, 11, 13	1:25 – 2:05
SHAP Interpretation	Notebook 4	Cells 2, 3, 4, 5, 7	2:05 – 2:40
Conclusion	—	Summary / README	2:40 – 3:00

Introduction & Problem Statement

0:00 – 0:20

Hi, I am [Your Name], and today I'll walk you through our Data Science project — a machine learning study that tackles a fundamental question: Can socioeconomic indicators like GDP, population, and energy usage predict global climate outcomes? We built an end-to-end ML pipeline covering 195 countries over 123 years — 1900 to 2023 — across four modular Jupyter notebooks. Let's dive in.

NOTEBOOK 1

Exploratory Data Analysis —

01_eda_visualizations.ipynb

0:20 – 0:55

We begin with Notebook 1 — Exploratory Data Analysis. The goal here is to understand the data's structure, assess its quality, tackle panel-balancing issues, and extract early insights about our two targets: Temperature Anomaly and CO₂ Emissions.

Cell 2 — Imports & Configuration

We load pandas, NumPy, seaborn, and scipy. Plotting styles are configured here — this cell is the environment setup ensuring all visualizations are consistent and reproducible.

Cell 3 — Data Load & Summary Statistics

This is our first look at the raw data. We load the dataset and immediately print: shape (rows × columns), data types, a full `df.describe()` statistical summary, missing value counts per column, and the number of unique countries and years.

Say: "At this point we have roughly 100,000 raw rows across 26 columns. The `describe()` output immediately flags skewed distributions — CO₂ emissions has a much larger std than mean, indicating heavy right skew from industrial outliers."

Cell 4 – Panel Integrity Check (Duplicates & Completeness)

This is where we tackle panel-balancing. We check for duplicate (Country, Year) pairs and compute panel completeness: Expected rows = 195 countries × 124 years = 24,180. Actual rows may differ — exposing gaps in historical coverage.

Say: "We discovered duplicate (Country, Year) entries — multiple records per country per year — a typical issue when merging datasets from different sources. We also found that the panel was unbalanced: not every country has data for every year, especially pre-1950."

Cell 5 – Records Per Group Distribution

We group by (Country, Year) and count how many input rows exist per group. The `value_counts()` output shows the frequency of duplicates — e.g., how many groups had 1 record vs. 2 vs. 3+.

Say: "Most groups had exactly 1 record; a subset had 2–3 due to source overlaps. This directly informed our aggregation strategy."

Cell 6 – Aggregation to Balance the Panel

We resolve duplicates here. All numeric columns are aggregated by taking the mean per (Country, Year) group, collapsing multiples into a single canonical row. After aggregation, the dataset becomes a clean, one-row-per-country-year panel.

Say: "By groupby-mean aggregation, we reduced the dataset from ~100K rows to ~23,797 rows — one row per (Country, Year) — giving us a balanced, structured panel ready for time-series modeling."

Cell 7 – Global Trends Over Time

A dual-axis time series plot: left axis shows Temperature Anomaly, right axis shows CO₂ Emissions. Overlaid with GDP and Population trends.

Say: "The diverging trends after 1950 are unmistakable — rapid industrial growth drives exponential CO₂ rise, while temperature anomaly climbs more gradually. This temporal pattern motivates the need for time-aware feature engineering."

Cell 11 – Full Correlation Heatmap

A heatmap of the full numeric correlation matrix. GDP, fossil fuel usage, population, and urbanization show strong positive correlations with CO₂. Temperature anomaly shows much weaker linear relationships with all features.

Cell 14 – EDA Summary & Key Findings

A printed structured summary confirming: final panel size (~23,797 rows), year range, country count, target statistics, and recommendations carried forward.

Say: "The EDA gave us two critical take-aways: CO₂ is strongly linearly predictable from socioeconomic data, while temperature anomaly is not — a finding our models would later confirm."

NOTEBOOK 2

Preprocessing Pipeline —

02_preprocessing_pipeline.ipynb

0:55 – 1:25

Notebook 2 is our preprocessing pipeline — converting the cleaned EDA output into a modeling-ready format without any data leakage.

Cell 2 – Imports & Directory Setup

We import pandas, sklearn's RobustScaler and TimeSeriesSplit, and joblib for serialization. Output directories for processed data and models are created here.

Say: "We chose RobustScaler specifically because economic data like GDP has extreme outliers — it scales using the interquartile range, making it outlier-resistant."

Cell 3 – Load Data & Temporal Train/Test Split

We load the cleaned CSV (Shape: 23,797 × 26). A **temporal split** is applied: Train = 1900–2009 (21,106 rows), Test = 2010–2023 (2,691 rows). No random shuffling allowed — this is a time series panel.

Say: "A strict temporal split is critical. If we randomly split, future data leaks into training — like predicting 1950 emissions using 2010 data. We train on the past and test on the most recent 14 years."

Cell 6 – Full Feature Engineering + Scaling + Save

The core preprocessing cell. Calls `engineer_full_pipeline()` from `src/feature_engineering.py` which generates 16 temporal features: lag variables, rolling means, energy intensity ratios, per-capita indicators, and policy-GDP interaction terms. Dataset grows from 26 → 47 columns. RobustScaler is fit on train only and applied to both train and test. Final artifacts saved: 4 CSV files + scaler pickle. Final shape: Train (20,131 × 20), Test (2,691 × 20).

Say: "All transformations — scaling, imputation, encoding — are fit on training data only and then applied to the test set. This is the gold standard for preventing leakage in time-series modeling."

 **Key Insight to Mention:** The scaler is saved as a pickle so any future inference on new data uses the exact same transformation fitted on 1900–2009 data — ensuring full reproducibility.

NOTEBOOK 3

Model Training & Tuning —

03_model_training_tuning.ipynb

1:25 – 2:05

Notebook 3 is where we train and compare five different ML architectures for multi-target regression — simultaneously predicting both Temperature Anomaly and CO₂ Emissions.

Cell 1 – Markdown: Objective

States the modeling objective: multi-target regression on two targets using MultiOutputRegressor wrappers and native multi-output models.

Cell 4 – Evaluation Function

`evaluate_multi_target()` computes R^2 , RMSE, and MAE for each target independently and as a macro-average. This function is used consistently across all models for fair comparison.

Say: "Having a single evaluation harness ensures apples-to-apples comparison — the same metrics computed the same way for every model."

Cell 5 – Random Forest Training

200 trees, `n_jobs=-1` (all CPU cores), `random_state=42`. Achieves near-perfect CO₂ $R^2 \approx 0.99$, but lower Temperature Anomaly R^2 .

Say: "Random Forest is our first strong baseline. The CO₂ performance is striking — almost perfect. Temperature anomaly is much harder to predict, which immediately tells us something scientifically important."

Cell 7 – XGBoost Training

200 estimators, `max_depth=4`, `learning_rate=0.1`, `subsample=0.8`. Comparable to Random Forest accuracy but significantly faster training time.

Cell 9 – LightGBM Training

Same config as XGBoost. Even faster thanks to histogram-based leaf-wise tree growth. GPU-compatible architecture.

Say: "We benchmarked three gradient-boosted tree models. All converge on similar accuracy — what differs is training speed and interpretability tradeoffs."

Cell 11 – Optuna Hyperparameter Tuning

Bayesian optimization with Optuna over the XGBoost search space for `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`. Uses `TimeSeriesSplit` cross-validation as the inner loop to prevent leakage during tuning.

Say: "Optuna is smarter than grid search — it learns which hyperparameter regions are promising and focuses trials there. We ran N trials and found the globally optimal XGBoost configuration."

Cell 13 — Final Model Comparison Chart

Bar chart comparing all models on R^2 , RMSE per target. XGBoost Tuned wins on the macro-average. The chart clearly shows the CO₂ vs Temperature Anomaly performance gap.

Say: "This summary chart is the key deliverable of Notebook 3. CO₂ R^2 is near 1.0 across all models. Temperature Anomaly R^2 is consistently lower — confirming the predictability asymmetry hypothesis."

NOTEBOOK 4

Model Interpretation — 04_interpretation.ipynb

2:05 – 2:40

Notebook 4 goes beyond accuracy — we use SHAP, SHapley Additive exPlanations, to understand WHY our best model makes specific predictions, and which socioeconomic factors matter most for each climate target.

Cell 2 — Load Best Model & SHAP Setup

Loads the saved XGBoost Tuned model from disk (joblib), loads the test set, and imports the SHAP library. Two separate models are loaded — one per target — since we wrapped with `MultiOutputRegressor`.

Cell 3 — Compute SHAP Values

shap.TreeExplainer is called for both the Temperature Anomaly and CO₂ Emissions sub-models. SHAP values are computed on the entire test set.

Say: "TreeExplainer is exact — not approximate — for tree-based models. It computes the contribution of every single feature to every single prediction in polynomial time."

Cell 4 – Global SHAP Summary Plots

get_feature_importance() computes mean absolute SHAP values per feature. Two SHAP beeswarm/bar plots are generated — one for each target — ranking features by global importance.

Say: "For CO₂: fossil fuel usage, population, and GDP dominate. For Temperature Anomaly: urbanization and lagged climate signals emerge. This asymmetry in driver structure is the paper's central finding."

Cell 5 – SHAP Dependence Plots

Marginal effect plots for the top drivers. Urbanization vs Temperature Anomaly shows a non-linear threshold — once urbanization crosses a certain level, its climate impact spikes disproportionately.

Say: "These dependence plots reveal threshold effects — non-linearities that a linear regression would completely miss. This is the real power of interpretable ML: quantifying when economic growth starts having disproportionate climate impact."

Cell 7 – Final Interpretation Summary

Printed output: CO₂ R² ≈ 0.99+, Temperature Anomaly R² significantly lower. Confirms the predictability asymmetry. Lists top 5 drivers per target.

Say: "The model summary confirms what the SHAP plots showed — socioeconomic data gives us a near-perfect lens into emissions, but temperature anomaly is governed by complex physical feedbacks that economics alone cannot capture."

Conclusion • 2:40 – 3:00

Wrap up by returning to the README or a summary slide.

Built an end-to-end ML pipeline on 123 years of global panel data across 195 countries.

Tackled panel-balancing through groupby aggregation and temporal train/test splitting.

Engineered 47 time-aware features including lags, rolling statistics, and interaction terms.

Benchmarked 5 models; selected XGBoost Tuned via Optuna as the best performer.

Used SHAP to deliver feature-level interpretability and discover non-linear threshold effects.

- **Key Finding:** CO_2 $R^2 \approx 0.99$ — but temperature anomaly remains harder to predict from socioeconomics alone, which is itself an important scientific result.

"Thank you — happy to take questions on any part of the pipeline."



Presentation Tips

- Pre-run all cells before recording so outputs and charts are pre-rendered — just scroll, no live execution needed.
- Keep Jupyter in presentation mode (View → Presentation) or zoom browser to 125% for readability.
- For each notebook, briefly show the file name in the Jupyter tab so the audience can see you moving across notebooks.
- Highlight outputs on screen (circle/zoom) rather than just talking — visual anchors keep attention.